데이터 증대

- 데이터 증대(data augmentation)는 한정된 데이터에서 여러 가지로 변형된 데이터를 만들어내는 기법이다.





라이브러리를 포함시킨다.

```
import tensorflow as tf
import matplotlib.pyplot as plt
from numpy import expand_dims
from tensorflow.keras.preprocessing.image import load_img, img_to_array

image = load_img("dog.jpg")
array = img_to_array(image)
sample = expand_dims(array, axis=0)
```



ImageDataGenerator()을 이용하여 영상 변형을 정의한다.

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(rescale = 1./255,
    rotation_range=90, brightness_range=[0.8, 1.0],
    width_shift_range=0.2, zoom_range=[0.8, 1.2],
    height_shift_range=0.2)
```

- rotation_range=90: 회전 한도
- brightness_range=[0.2, 1.0]: 밝기 변형 비율
- width_shift_range=0.2: 좌우 이동 한도
- zoom_range=[0.2, 1.2]: 확대 한도



ImageDataGenerator로부터 제너레이터 객체를 얻는다.

- ImageDataGenerator 출력은 파이썬의 제너레이터 형식이다.
- 영상을 생성하는 제너레이터 객체를 생성한다.
- 제너레이터는 next()가 호출되면 변형된 영상을 하나씩 보내게 된다.

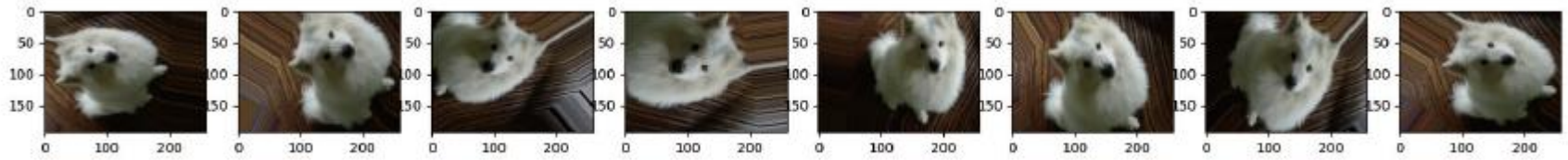
```
obj = datagen.flow(sample, batch_size=1)
```



변형된 영상을 표시한다.

```
obj = datagen.flow(sample, batch_size=1)
fig = plt.figure(figsize=(20,5))

for i in range(8):
  plt.subplot(1,8,i+1)
  image = obj.next()
  plt.imshow(image[0])
```





가중치 저장과 전이 학습

- 우리는 이미 학습된 모델의 가중치를 저장할 수 있고, 필요할 때마다 가중치를 불러와서 바로 신경망이 예측을 할 수 있게 할 수 있다.





가중치 저장과 복원

- 저장: model.save('mymodel.확장자')
- 저장되는 정보
  - 신경망 모델의 아키텍처 및 구성
  - 훈련 중에 학습된 모델의 가중치 값
  - 신경망 모델의 컴파일 정보
  - 옵티마이저와 현재 상태(훈련을 중단한 곳에서 다시 시작하기 위해)
- 저장 형식
  - Keras v3 >> mymodel.keras
  - H5 형식. >> mymodel.h5
- 복원: model = load_model('mymodel. 확장자 ')



예제

```
import numpy as np
import tensorflow as tf

# 난수로 훈련 예제를 만든다.
test_input = np.random.random((128, 32))
test_target = np.random.random((128, 1))

# 입력이 32, 출력이 1 노드인 신경망 모델을 구축한다. 함수형 API를 사용하였다.
inputs = tf.keras.Input(shape=(32,))
outputs = tf.keras.layers.Dense(1)(inputs)
model = tf.keras.Model(inputs, outputs)
model.compile(optimizer="adam", loss="mean_squared_error")

# 신경망을 3번 훈련시킨다.
model.fit(test_input, test_target, epochs=3)

# 3의 에포크를 수행한 모델을 저장한다.
model.save("my_model")

# 저장된 모델을 불러온다.
saved_model = tf.keras.models.load_model("my_model")

# 저장된 모델을 다시 학습시킨다.
saved_model.fit(test_input, test_target, epochs=3)
```

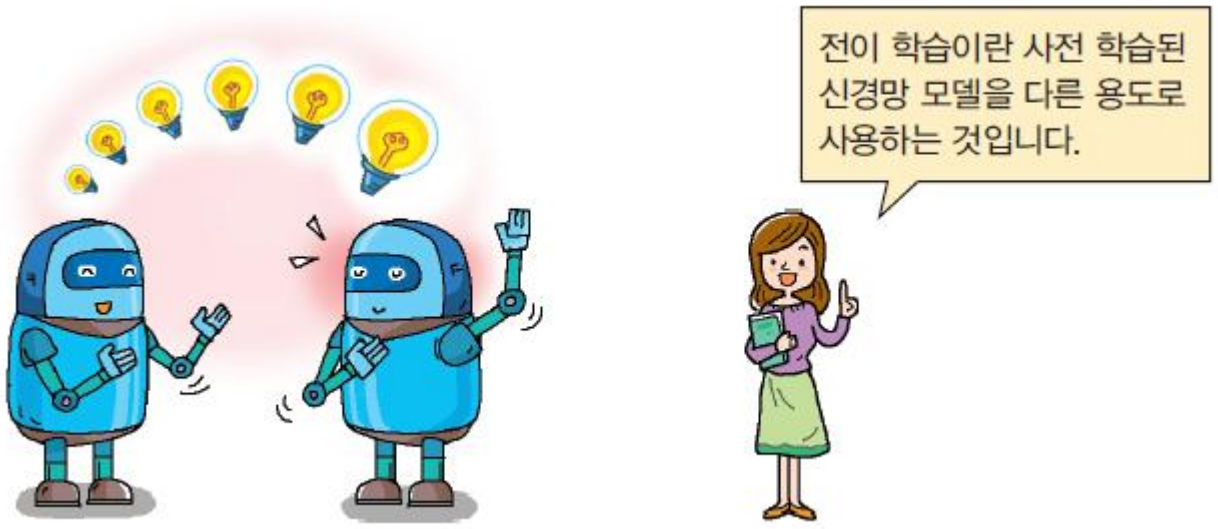


실행결과

```
Epoch 1/3
4/4 [==============================] - 0s 499us/step - loss: 0.7929
Epoch 2/3
4/4 [==============================] - 0s 499us/step - loss: 0.7068
Epoch 3/3
4/4 [==============================] - 0s 498us/step - loss: 0.6226
INFO:tensorflow:Assets written to: my_model\assets
Epoch 1/3
4/4 [==============================] - 0s 748us/step - loss: 0.5518
Epoch 2/3
4/4 [==============================] - 0s 564us/step - loss: 0.4866
Epoch 3/3
4/4 [==============================] - 0s 505us/step - loss: 0.4354
```

전이학습

- 전이 학습(transfer learning)은 하나의 문제에 대해 학습한 신경망의 모델과 가중치를, 새로운 문제에 적용하는 것이다.



전이 학습

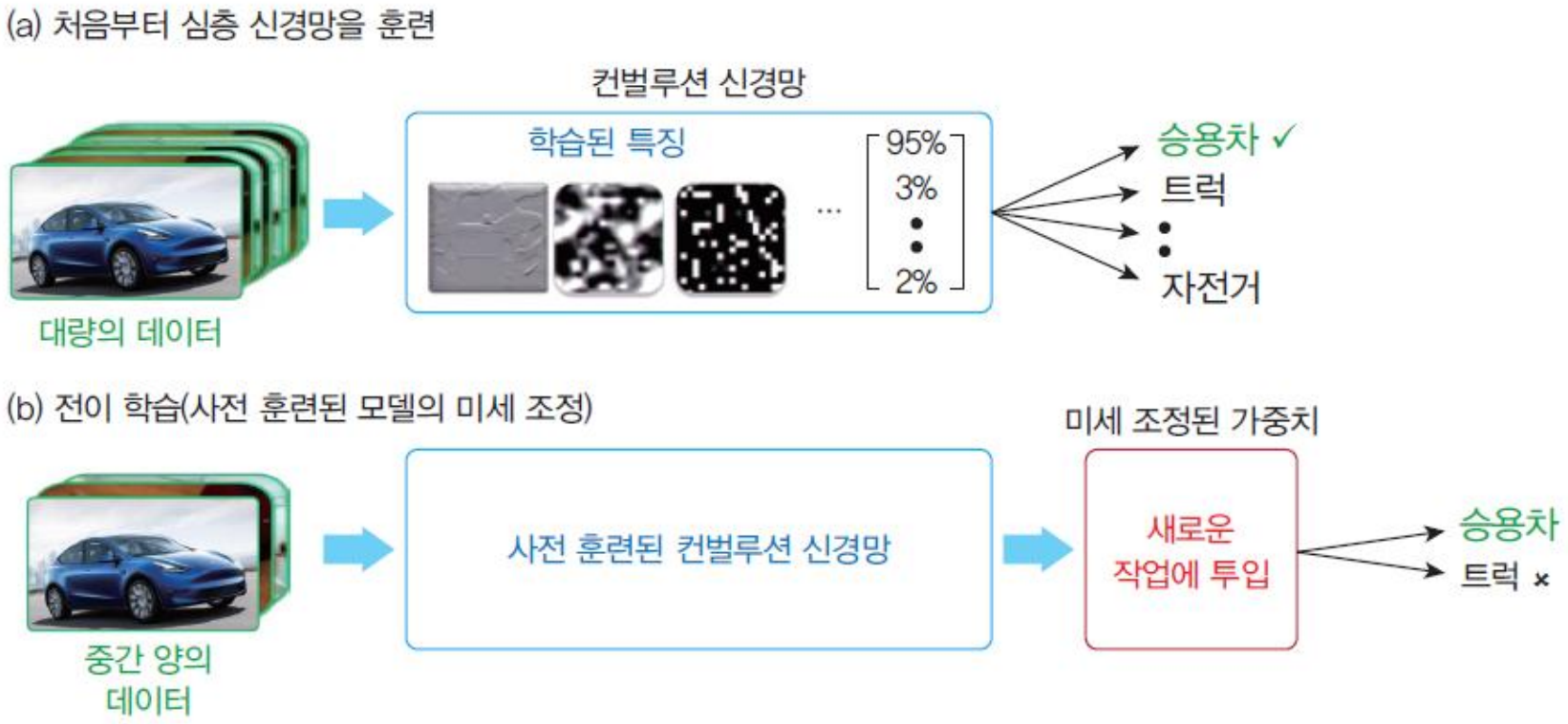


그림 10-12 (a) 처음부터 개발하는 방법 (b) 사전 훈련된 모델을 받아서 미세 조정하는 방법



사전 훈련된 신경망 모델

- 케라스는 사전 훈련된 딥러닝 모델들을 제공한다. 이것을 케라스에서는 “케라스 애플리케이션(keras applications)”이라고 부른다

모델	크기	Top-1 정확도	Top-5 정확도	매개 변수	깊이
Xception	88MB	0.790	0.945	22,910,480	126
VGG16	528MB	0.713	0.901	138,357,544	23
VGG19	549MB	0.713	0.900	143,667,240	26
ResNet50	98MB	0.749	0.921	25,636,712	–
ResNet101	171MB	0.764	0.928	44,707,176	–
ResNet152	232MB	0.766	0.931	60,419,944	–
ResNet50V2	98MB	0.760	0.930	25,613,800	–
ResNet101V2	171MB	0.772	0.938	44,675,560	–
ResNet152V2	232MB	0.780	0.942	60,380,648	–
InceptionV3	92MB	0.779	0.937	23,851,784	159
InceptionResNetV2	215MB	0.803	0.953	55,873,736	572
MobileNet	16MB	0.704	0.895	4,253,864	88
MobileNetV2	14MB	0.713	0.901	3,538,984	88



사전 훈련된 모델을 내 프로젝트에 맞게 재정의하기

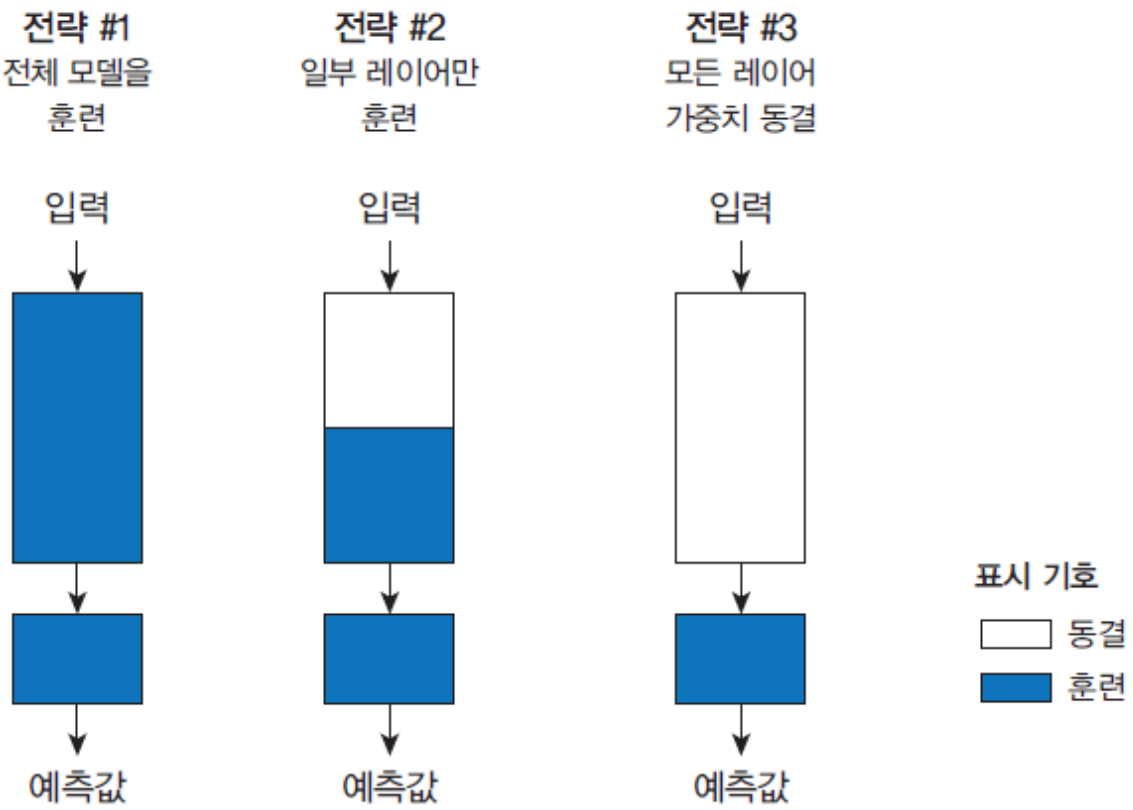


그림 10-13 사전 훈련 모델을 사용하는 3가지 방법



예제 #1

- 첫 번째 예제는 ResNet50을 다운로드받아서 변경하지 않고 그대로 사용해보자. 구체적으로 인터넷에서 강아지 사진을 다운받아서 올바르게 인식하는지를 보자





```
from tensorflow.keras.applications.resnet50 import ResNet50
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.resnet50 import preprocess_input,
decode_predictions
import numpy as np

model = ResNet50(weights='imagenet')

img_path = 'dog.jpg'
img = image.load_img(img_path, target_size=(224, 224))  # 영상 크기를 변경하고
적재한다.
x = image.img_to_array(img)              # 영상을 넘파이 배열로 변환한다.
x = np.expand_dims(x, axis=0)            # 차원을 하나 늘인다. 배치 크기가 필요하다.
x = preprocess_input(x)       # ResNet50이 요구하는 전처리를 한다.

preds = model.predict(x)
print('예측:', decode_predictions(preds, top=3)[0])
```

```
예측: [('n02111889', 'Samoyed', 0.9557966), ('n02114548', 'white_wolf', 0.01857086),
('n02112018', 'Pomeranian', 0.00947881)]
```



Summary

- 영상 인식(image recognition)란 영상 안의 물체를 인식하거나 분류하는 것이다.
- 영상 인식에 많이 사용되는 신경망은 컨볼루션 인공신경망(CNN)이다. 컨볼루션 신경망(CNN)은 동물의 조직에서 영감을 얻어서 만들어진 신경망이다. CNN에서 뉴런의 수용 공간은 다른 뉴론들과 약간 겹치게 된다. 컨볼루션 신경망은 영상 및 비디오 인식, 추천 시스템 및 자연 언어 처리 분야에서 폭넓게 응용되고 있다.
- 풀링(Pooling)이란 서브 샘플링이라고도 하는 것으로 입력 데이터의 크기를 줄이는 것이다.
- 데이터 증대(data augmentation)는 한정된 데이터에서 여러 가지로 변형된 데이터를 만들어내는 기법이다.
- 케라스에서는 save(), load_model() 메소드를 이용하여 가중치를 저장하거나 불러올 수 있다.
- **전이 학습(transfer learning)**은 하나의 문제에 대해 학습한 신경망의 모델과 가중치를, 새로운 문제에 적용하는 것이다.